Actividad 11 — Estructura de base de datos MySQL

Objetivo y entregables

Objetivo: Definir e implementar la base de datos del proyecto Sketch & Coffee.

La consigna de la actividad solicita:

1. Crear en MySQL Workbench el diagrama entidad-relación del proyecto con al menos cuatro entidades.
2. Ejecutar los scripts de creación de la base de datos y sus tablas sin errores.
3. Crear scripts SQL para insertar cinco registros de muestra en cada tabla.
4. Entregar el diagrama y los scripts listos para ejecutarse.

Criterios de aceptación: el diagrama contiene cuatro o más entidades; el script crea la base de datos y sus tablas sin errores; los inserts se ejecutan sin errores y cada tabla queda con cinco registros de prueba (5 usuarios - 5 productos - 5 categorías).

Revisión de los datos existentes

Se revisaron todos los archivos .json del repositorio:

- **menu/JSON documentos/menu.json**: Catálogo de 140 productos en formato de exportación de un sistema externo. Incluye departamento, nombre, SKU, existencias, precios dentro de origins y opciones. Fuente principal para seleccionar y transformar productos. No copiarlo directamente a una tabla: primero normalizar los campos y revisar SKU, precios y existencias.
- **menu_interactivo/menu.json**: Catálogo para la interfaz, organizado por categorías y subcategorías. Contiene 33 productos, todos bajo Bebidas; Postres, Platillos y Crepas y Waffles están vacíos. Referencia para nombres, categorías y estructura sencilla; no representa todavía todo el menú.
- **.vscode/settings.json**: Configuración del puerto de Live Server. No contiene datos del negocio.
- **pruebasUnitariasValidacion/package.json**: Configuración de Jest y scripts de pruebas. No contiene datos del negocio.
- **pruebasUnitariasValidacion/package-lock.json**: Versiones bloqueadas de dependencias de pruebas. No contiene datos del negocio.

Conclusión sobre usuarios: no existía un archivo JSON de usuarios. El proyecto sí registra usuarios en localStorage; el registro captura nombre, apellido, teléfono y correo, y el código añade identificador y rol. La contraseña que usa la aplicación actual no debe copiarse a datos de muestra ni guardarse como texto plano en MySQL. Se agregó menu/JSON documentos/usuarios.json con cinco perfiles completamente ficticios y sin credenciales.

Decisión de alcance: para esta actividad, tomar el catálogo de 140 productos como fuente de referencia y usar usuarios.json como datos de prueba de usuarios. El equipo debe acordar una sola convención para categorías y precios; en el JSON grande, los precios están dentro de origins y hay existencias negativas, así que no deben importarse sin validación.
Mantener los archivos de origen sin cambios.

Modelo relacional propuesto

Se propone la implementación de un esquema relacional conformado por **cinco entidades principales**. Este diseño excede el mínimo requerido de cuatro entidades y satisface plenamente la lógica transaccional de *Sketch & Coffee*, permitiendo registrar clientes, organizar el catálogo por categorías y procesar órdenes de compra con su desglose detallado de ítems.

Orden de creación y dependencias de las tablas :

Creación de la base de datos

```
CREATE DATABASE IF NOT EXISTS sketch_coffee
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE sketch_coffee;
```

Nombre de la tabla: **Categorias**

```
-- Nombre de la tabla: Categorias
CREATE TABLE categorias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(300) NULL
);

INSERT INTO categorias (id, nombre, descripcion) VALUES
     (1, 'Bebidas calientes', 'Bebidas calientes del menú interactivo.'),
    (2, 'Bebidas frías', 'Bebidas frías del menú interactivo.'),
    (3, 'Postres', 'Postres y productos de panadería.'),
    (4, 'Platillos', 'Alimentos salados del menú.'),
    (5, 'Café en grano', 'Café de especialidad en distintas presentaciones.');
```

Nota: Las categorías son las que se encuentran en el proyecto.

Nombre de la tabla: **Pedidos**

```
CREATE TABLE pedidos (
id INT AUTO_INCREMENT PRIMARY KEY,
usuario_id INT NOT NULL,
fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
estado  ENUM('pendiente',  'pagado',  'en_preparacion',  'entregado',  'cancelado')
DEFAULT 'pendiente',
total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
CONSTRAINT fk_pedidos_usuario FOREIGN KEY (usuario_id)
REFERENCES usuarios(id)
ON DELETE CASCADE
ON UPDATE CASCADE
)
```

Nombre de la tabla (clientes y admin): **Usuarios**

```
CREATE TABLE usuario (
    Id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    telefono VARCHAR(20) NULL,
    correo VARCHAR(100) NOT NULL UNIQUE,
    rol ENUM('cliente', 'admin') DEFAULT 'cliente',
    activo BOOLEAN DEFAULT TRUE,
    password_hash VARCHAR(255) NULL
);
```

Nombre de la tabla: **Productos**

```
CREATE TABLE productos (
        id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
        categoria_id INT NOT NULL,
        sku VARCHAR(30) NOT NULL,
        nombre VARCHAR(100) NOT NULL,
        descripcion TEXT NULL,
        precio DECIMAL(10, 2) NOT NULL,
        existencias INT NOT NULL DEFAULT 0,
        activo BOOLEAN NOT NULL DEFAULT TRUE,

        PRIMARY KEY (id),
        CONSTRAINT uq_productos_sku UNIQUE (sku),
        CONSTRAINT fk_productos_categoria
                FOREIGN KEY (categoria_id) REFERENCES categorias(id)
                ON DELETE RESTRICT
                ON UPDATE CASCADE,
        CONSTRAINT chk_productos_precio CHECK (precio >= 0),
        CONSTRAINT chk_productos_existencias CHECK (existencias >= 0)
        )
ENGINE = InnoDB
DEFAULT CHARSET = utf8mb4
COLLATE = utf8mb4_unicode_ci;
```

```
INSERT INTO productos (categoria_id, sku, nombre, descripcion, precio, existencias, activo) VALUES
  (1, '520015364',     'Capuchino Caliente',
```

```
    'Espresso con leche vaporizada y espuma cremosa.', 70.00,  40, TRUE),

 (2, '1052001536134', 'Chai Latte Frio',
    'Te chai con leche y hielo, con canela y cardamomo.', 80.00,  25, TRUE),

 (3, '1052001536198', 'Croissant Dulce',
    'Hojaldre horneado el mismo dia, con mantequilla.', 110.00,  15, TRUE),

 (4, '52001536118',   'Panque de Limon',
    'Panque casero de limon con glaseado.', 65.00,  12, TRUE),

 (5, '1052001536165', 'Veracruz Bolsa Cafe en Grano',
    'Cafe de altura de Veracruz, tostado medio, bolsa de 1 kg.', 320.00,  8, TRUE);
```

Nombre de la tabla **Agregar los Registros Pedidos**

```
INSERT INTO pedidos (usuario_id, fecha, estado, total) VALUES
(1, NOW(), 'pendiente', 450.00),
(2, NOW(), 'pagado', 1250.50),
(3, NOW(), 'en preparación', 90.00),
(4, NOW(), 'entregado', 320.00),
(5, NOW(), 'cancelado', 150.00);
```

Nombre de la tabla: **Detalles_pedidos**

```
CREATE TABLE detalles_pedido (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pedido_id INT NOT NULL,
    producto_id INT NOT NULL,
    cantidad INT NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,

    -- Restricciones de Clave Foránea
    /*
   CONSTRAINT-> Es una regla de seguridad que le impones a la base de datos
    para garantizar que los datos sean correctos, válidos y coherentes
    */
    CONSTRAINT fk_detalles_pedido FOREIGN KEY (pedido_id)
        REFERENCES pedidos(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_detalles_producto FOREIGN KEY (producto_id)
        REFERENCES productos(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

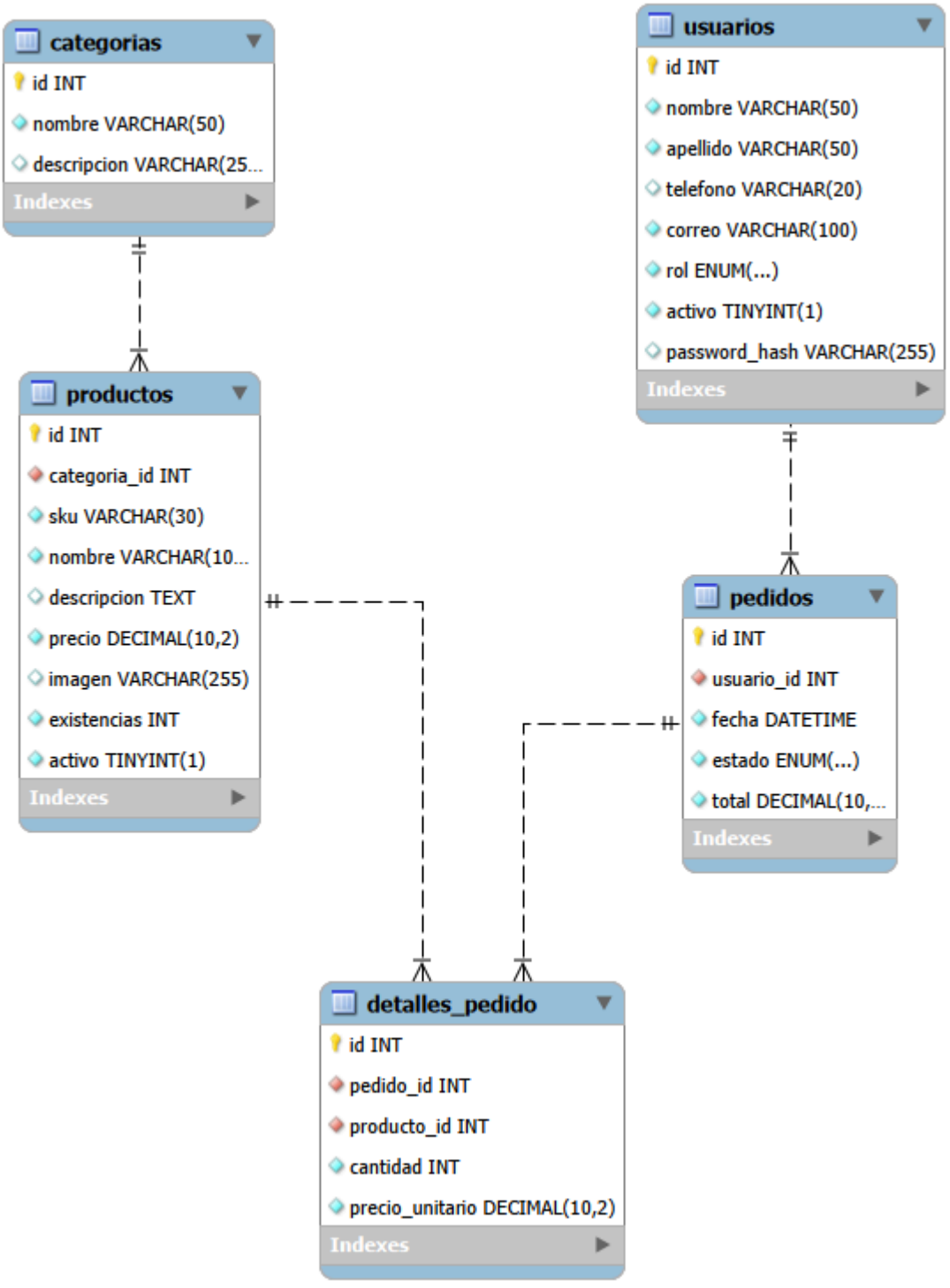
    -- Reglas de Integridad de Negocio
```

```
    /*
        CHECK-> Sirve para validar que los datos cumplan con una condición específica
    antes de permitir que se guarden en la tabla.
    */
    CONSTRAINT chk_cantidad_positiva CHECK (cantidad > 0),
    CONSTRAINT chk_precio_unitario_valido CHECK (precio_unitario >= 0)
);

INSERT INTO detalles_pedido (pedido_id, producto_id, cantidad, precio_unitario) VALUES
    (1, 1, 2, 70.00),   -- 2x Capuchino caliente ($70.00 c/u)
    (2, 2, 1, 80.00),   -- 1x Chai latte frío ($80.00)
    (3, 3, 1, 65.00),   -- 1x Panqué de limón ($65.00)
    (4, 4, 1, 130.00),  -- 1x Chilaquiles ($130.00)
    (5, 5, 1, 320.00);  -- 1x Veracruz bolsa de café en grano ($320.00)
```

1. Diagrama de Relaciones y Cardinalidades

- **categorias** (1) —— (N) **productos**
    - *Cardinalidad:* Uno a Muchos (1:N).
    - *Descripción:* Una categoría agrupa múltiples productos (ej. Bebidas, Postres), pero cada producto pertenece estrictamente a una sola categoría.
    - *Relación:* productos.categoria_id (FK) ➔ categorias.id (PK).
- **usuarios** (1) —— (N) **pedidos**
    - *Cardinalidad:* Uno a Muchos (1:N).
    - *Descripción:* Un usuario cliente puede realizar múltiples pedidos a lo largo del tiempo. Cada pedido está asociado a un único usuario responsable de la compra.
    - *Relación:* pedidos.usuario_id (FK) ➔ usuarios.id (PK).
- **pedidos** (1) —— (N) **detalles_pedido**
    - *Cardinalidad:* Uno a Muchos (1:N).
    - *Descripción:* Un pedido contiene uno o más ítems/líneas en su desglose. Cada registro de detalle pertenece a un único encabezado de pedido.
    - *Relación:* detalles_pedido.pedido_id (FK) ➔ pedidos.id (PK).
- **productos** (1) —— (N) **detalles_pedido**
    - *Cardinalidad:* Uno a Muchos (1:N).
    - *Descripción:* Un producto del catálogo puede figurar en los detalles de múltiples pedidos. Cada línea de detalle referencia a un solo producto específico.
    - *Relación:* detalles_pedido.producto_id (FK) ➔ productos.id (PK).



2. Especificación Técnica de las Entidades y Reglas de Integridad

1. **categorias** (Tabla de catálogo base)

- **Atributos:** id (INT, PK, AUTO_INCREMENT), nombre (VARCHAR(50), NOT NULL, UNIQUE), descripcion (VARCHAR(255), NULL).
- **Reglas de Integridad:** El campo nombre es de valor único obligatorio para evitar categorías duplicadas en la navegación.

2. **usuarios** (Tabla de clientes y personal)
   - **Atributos:** id (INT, PK, AUTO_INCREMENT), nombre (VARCHAR(50), NOT NULL), apellido (VARCHAR(50), NOT NULL), telefono (VARCHAR(20), NULL), correo (VARCHAR(100), NOT NULL, UNIQUE), rol (ENUM('cliente', 'admin'), DEFAULT 'cliente'), activo (BOOLEAN, DEFAULT TRUE), password_hash (VARCHAR(255), NULL).
   - **Reglas de Integridad:** correo debe poseer restricción UNIQUE para credenciales e identificación. Las contraseñas en entornos reales se gestionan mediante hash de seguridad (bcrypt/argon2).
3. **productos** (Tabla de catálogo de artículos)
   - **Atributos:** id (INT, PK, AUTO_INCREMENT), categoria_id (INT, NOT NULL, FK), sku (VARCHAR(30), NOT NULL, UNIQUE), nombre (VARCHAR(100), NOT NULL), descripcion (TEXT, NULL), precio (DECIMAL(10,2), NOT NULL), existencias (INT, DEFAULT 0), activo (BOOLEAN, DEFAULT TRUE).
   - **Reglas de Integridad:** precio y existencias deben validarse mediante restricción CHECK (precio >= 0 AND existencias >= 0). categoria_id establece la dependencia formal con la tabla categorias.
4. **pedidos** (Tabla de encabezado de transacciones)
   - **Atributos:** id (INT, PK, AUTO_INCREMENT), usuario_id (INT, NOT NULL, FK), fecha (DATETIME, DEFAULT CURRENT_TIMESTAMP), estado (ENUM('pendiente', 'pagado', 'en_preparacion', 'entregado', 'cancelado'), DEFAULT 'pendiente'), total (DECIMAL(10,2), NOT NULL, DEFAULT 0.00).
   - **Reglas de Integridad:** usuario_id asegura la trazabilidad del cliente en MySQL mediante clave foránea hacia usuarios(id).
5. **detalles_pedido** (Tabla intermedia / relación N:M resuelta)
   - **Atributos:** id (INT, PK, AUTO_INCREMENT), pedido_id (INT, NOT NULL, FK), producto_id (INT, NOT NULL, FK), cantidad (INT, NOT NULL), precio_unitario (DECIMAL(10,2), NOT NULL).
   - **Reglas de Integridad:** Resuelve la relación Muchos a Muchos entre pedidos y productos. Incluye restricción CHECK (cantidad > 0 AND precio_unitario >= 0) e historial del precio al momento de la venta.

3. Orden de Ejecución e Inserción de Datos (Integridad Referencial)

Para evitar violaciones de clave foránea en los scripts de creación (DDL) e inserción (DML), se debe respetar estrictamente el siguiente orden secuencial de operaciones:

1. categorias / usuarios ➔ 2. productos ➔ 3. pedidos ➔ 4. detalles_pedido

- **Estrategia DDL/DML:**
    - 1. Insertar primero los catálogos base (categorias y usuarios).
    - 2. Insertar productos asociando sus respectivas categoria_id.
    - 3. Registrar el encabezado de la compra en pedidos vinculando el usuario_id.
    - 4. Poblar detalles_pedido enlazando el pedido_id y producto_id correspondientes.

4. Validación Operativa, Ejecución de Consultas y Evidencias

Como parte de la fase de control de calidad, ejecución y aseguramiento de la integridad transaccional del proyecto Sketch & Coffee, se validó la correcta ejecución de los datos mediante la base de datos en MySQL Workbench. El script unificado se ejecutó de manera limpia garantizando la creación de las tablas, la inexistencia de errores de dependencias y el cumplimiento estricto del criterio de aceptación de contar con un mínimo de cinco registros por cada entidad principal (categorias, usuarios, pedidos y detalles_pedidos).

A continuación se presentan las evidencias documentales de la ejecución:

4.1. Registros en las tablas

Se ejecutaron consultas de conteo y muestreo individual por tabla para corroborar que los datos de prueba iniciales se insertaron de forma correcta, respetando los tipos de datos definidos:

- Conteo de filas: Se comprobó exitosamente que las cinco tablas principales contienen exactamente cinco registros activos de prueba, cumpliendo la métrica solicitada.

tabla	filas
categorias	5
usuarios	5
productos	5
pedidos	5
detalles_pedido	5

- Categorías: Validación del catálogo base donde se observan las 5 categorías requeridas (Bebidas frías, Bebidas calientes, Postres, Platillos y Café en grano) con sus respectivas descripciones.

id	nombre	descripcion
1	Bebidas frías	Frappés, sodas italianas, tisanas heladas y bub…
2	Bebidas calientes	Cafés, tés y tisanas preparados al momento.
3	Postres	Panqués, flanes, gelatinas y panadería dulce.
4	Platillos	Molletes, chapatas, chilaquiles y alimentos salad…
5	Café en grano	Café de especialidad en bolsa, molido o en grano.
NULL	NULL	NULL

- Usuarios: Comprobación del registro de clientes de prueba y perfil administrativo, verificando la unión del correo electrónico y el resguardo de credenciales sin texto plano.

id	nombre	apellido	correo	rol	activo
1	Ana	Ramírez	ana.ramirez@ejemplo.com	cliente	1
2	Carlos	Mendoza	carlos.mendoza@ejemplo.com	cliente	1
3	Daniela	Ortiz	daniela.ortiz@ejemplo.com	cliente	1
4	Miguel	Hernández	miguel.hernandez@ejemplo.com	cliente	0
5	Sketch	Admin	admin@sketchandcoffee.mx	admin	1
NULL	NULL	NULL	NULL	NULL	NULL

- Productos: Validación de los artículos del menú, corroborando sus vinculación a categorías (categoria_id), clave SKU, precios unitarios y control de existencias.

id	categoria_id	sku	nombre	descripcion	precio	imagen	existencias	activo
1	2	520015364	Capuchino caliente	Espresso con leche vaporizada y espuma cremo…	70.00	NULL	40	1
2	1	1052001536134	Chai latte frío	Té chai con leche y hielo, con canela y cardamo…	80.00	NULL	25	1
3	3	52001536118	Panqué de limón	Panqué casero de limón con glaseado.	65.00	NULL	12	1
4	4	1052001536174	Chilaquiles	Totopos bañados en salsa de la casa, con crem…	130.00	NULL	18	1
5	5	1052001536165	Veracruz bolsa de café en grano	Café de altura de Veracruz, tostado medio, bols…	320.00	NULL	8	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Pedidos: Verificación de las órdenes de compra registradas, validando las fechas, los estados del pedido (entregado, pagado, en_preparacion, pendiente, cancelado) y el total guardado.

id	usuario_id	fecha	estado	total
1	1	2026-10-20 09:15:00	entregado	140.00
2	2	2026-10-21 11:40:00	pagado	320.00
3	3	2026-10-22 16:05:00	en_preparacion	130.00
4	1	2026-10-23 08:30:00	pendiente	160.00
5	4	2026-10-23 10:00:00	cancelado	65.00
NULL	NULL	NULL	NULL	NULL

- Detalles de Pedido: Inspección de la tabla intermedia que desglose los productos por orden de compra, verificando las llaves foráneas (pedido_id, producto_id), la cantidad solicitada y el precio unitario.

id	pedido_id	producto_id	cantidad	precio_unitario
1	1	1	2	70.00
2	2	5	1	320.00
3	3	4	1	130.00
4	4	2	2	80.00
5	5	3	1	65.00
NULL	NULL	NULL	NULL	NULL

4.2. Pruebas de que las relaciones y los totales funcionan

Para demostrar la correcta operatividad del esquema relacional en escenarios transaccionales reales, se ejecutaron consultas de comprobación:

- Relación de pedidos con clientes y productos: Se implementó una consulta utilizando operadores de combinación (INNER JOIN) para unir la información del pedido, el nombre del cliente, el producto adquirido, la categoría a la que pertenece, la cantidad y el importe. Esto permite constatar que las llaves foráneas (FK) enlazan de manera íntegra la información transaccional sin pérdida de datos.

pedido	cliente	producto	categoria	cantidad	precio_unitario	importe	estado
1	Ana Ramírez	Capuchino caliente	Bebidas calientes	2	70.00	140.00	entregado
2	Carlos Mendoza	Veracruz bolsa de café en grano	Café en grano	1	320.00	320.00	pagado
3	Daniela Ortiz	Chilaquiles	Platillos	1	130.00	130.00	en_preparacion
4	Ana Ramírez	Chai latte frío	Bebidas frías	2	80.00	160.00	pendiente
5	Miguel Hernández	Panqué de limón	Postres	1	65.00	65.00	cancelado

- Revisión de sumas y totales: Se realizó una verificación matemática comparando el campo total_guardado en el encabezado del pedido contra la suma_renglones calculada a partir de los detalles. La consulta arroja un estado de validación OK en la columna cuadra para la totalidad de los registros, garantizando la congruencia contable y de negocio del sistema.

pedido	total_guardado	suma_renglones	cuadra
1	140.00	140.00	OK
2	320.00	320.00	OK
3	130.00	130.00	OK
4	160.00	160.00	OK
5	65.00	65.00	OK

5. Conclusión del Informe Final

Para concluir, podemos decir que el trabajo con la base de datos Skech & Coffee salió excelente. Logramos ejecutar en My SQL Workbench el script completo que junta la parte DDL- es decir, creación de nuestras cinco tablas principales (categorias, usuarios, productos, pedidos y detalles_pedido) con todas sus reglas y llaves foráneas- y la parte DML, que fue la encargada de meter los cinco registros de prueba limpios en cada una.

Además, las consultas que hicimos nos sirvieron para comprobar que todo funciona de forma correcta en la práctica. Con el INNER JOIN confirmamos que los datos se concretan perfectamente entre clientes, productos y pedidos, y con la prueba de totales pudimos verificar que las cuentas y sumas cuadran a la perfección con un estado OK. Estando lista para conectarse con el sistema web de la ecommerce.

Distribución del equipo (7 integrantes)

Se asignaron nombres a todos los compañeros y se reestructuró la división del trabajo para eliminar tareas duplicadas, repartir formalmente cada tabla e incorporar expresamente la responsabilidad del diagrama ER:

- **Vanya — Product Owner (PO) + Tablas usuarios/admin y categorias:**
  - *Responsabilidad:* Aceptación final contra los criterios del proyecto, priorizar alcance y elaborar DDL y 5 inserts para las tablas usuarios y categorias.
  - *Entregable:* DDL, 5 inserts por tabla y validación de aceptación del proyecto.
- **Gonzalo — Scrum Master (SM) + Tabla productos:**
  - *Responsabilidad:* Facilitar acuerdos del equipo, cuidar el cumplimiento del timebox, resolver bloqueos en Trello y elaborar DDL y 5 inserts para la tabla productos.
  - *Entregable:* Tablero actualizado, timebox controlado, DDL y 5 inserts de productos.
- **Laura — Developers - Tabla pedidos:**
  - *Responsabilidad:* Definir y probar la tabla pedidos (encabezados de transacciones), configurando clave foránea usuario_id, estados y total.
  - *Entregable:* DDL y 5 inserts de la tabla pedidos.
- **Manuel — Developers - Tabla detalles_pedido:**
  - *Responsabilidad:* Definir y probar la tabla detalles_pedido, resolviendo la relación N:M con sus dos llaves foráneas (pedido_id y producto_id), cantidad y precio unitario.
  - *Entregable:* DDL y 5 inserts de detalles_pedido.
- **Luis Fernando — Developers - Diagrama ER:**
  - *Responsabilidad:* Diseñar y construir el diagrama Entidad-Relación en MySQL Workbench con cardinalidades, PK, FK y atributos completos, y exportarlo en formato imagen/PDF.
  - *Entregable:* Archivo .mwb y diagrama ER exportado.
- **Lizeth — Developers - Integración y Pruebas SQL:**
  - *Responsabilidad:* Integrar todos los scripts DDL y DML de los compañeros en un solo script ejecutable unificado y probarlo en una base de datos limpia para garantizar cero errores de dependencias.
  - *Entregable:* Script SQL final unificado y funcionando sin errores.
- **Alexandra — Developers - Evidencias e Informe Final:**
  - *Responsabilidad:* Ejecutar las consultas de comprobación (SELECT con las 5 filas por tabla), recopilar capturas de pantalla de evidencia y estructurar el informe final del equipo.

- *Entregable:* Informe/entregable final estructurado con evidencias de ejecución.

Timebox sugerido: 4 horas

Definición de Hechos

- Diagrama ER de MySQL Workbench con al menos cuatro entidades y relaciones legibles.
- Script de creación de base de datos y tablas ejecutado sin errores en una base limpia.
- Todas las tablas tienen PK; las relaciones tienen FK y restricciones coherentes.
- Script de datos con cinco registros por cada tabla, ejecutado sin errores y respetando las FK.
- Consultas de comprobación muestran cinco filas en cada tabla.
- El diagrama y los scripts entregados coinciden entre sí y usan los mismos nombres de tablas y columnas.
- No se incluyeron contraseñas reales ni texto plano de contraseña en los datos de muestra.

Acuerdos para agilizar y reducir retrabajo

1. Hacer primero el acuerdo de entidades y nombres; no desarrollar cada tabla con convenciones distintas.
2. Trabajar en paralelo después de acordar PK, FK, tipos y nombres. Integrar cambios en un único script; no entregar scripts incompatibles.
3. Probar temprano con una base vacía y ejecutar los inserts en orden de dependencias.
4. Usar datos claramente ficticios y conservar intactos los JSON existentes.
5. Mantener una sola fuente de verdad para el diagrama y el esquema final; cada cambio de columna debe actualizar ambos.
6. El PO acepta contra los criterios de la tarea y el SM mantiene el flujo, el tiempo y la visibilidad de bloqueos.